

Lab 8

Image Classification using Convolutional Neural Networks (CNN)

Exercise 1: Dataset Loading

```
In [2]: import tensorflow as tf
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print(f"Training data shape: {x_train.shape}")
print(f"Testing data shape: {x_test.shape}")
print(f"Number of classes: {len(set(y_train))}")
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 0s 0us/step

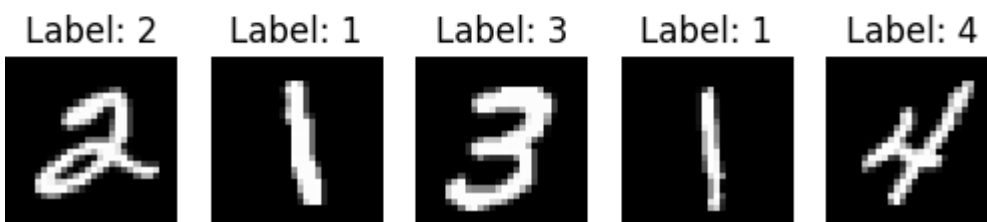
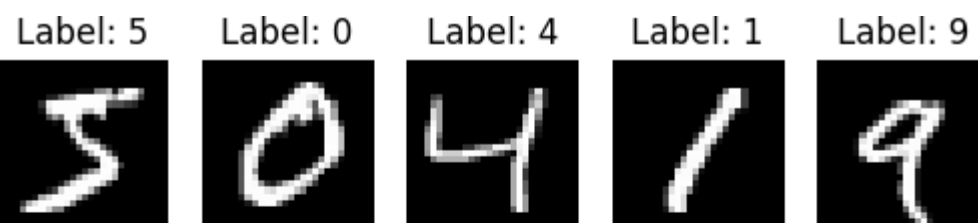
Training data shape: (60000, 28, 28)

Testing data shape: (10000, 28, 28)

Number of classes: 10

Exercise 2: Data Visualization

```
In [3]: import matplotlib.pyplot as plt
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis('off')
plt.show()
```



Exercise 3: Data Preprocessing

```
In [5]: import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
print(f"Shape of x_train: {x_train.shape}")
print(f"Shape of x_test: {x_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

```
Shape of x_train: (60000, 28, 28, 1)
Shape of x_test: (10000, 28, 28, 1)
Shape of y_train: (60000, 10)
Shape of y_test: (10000, 10)
```

Exercise 4: CNN Model Architecture

```
In [6]: from tensorflow.keras.models import Sequential
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D((2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Exercise 5: Model Compilation

```
In [13]: model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Exercise 6: Model Training

```
In [8]: history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.1)
```

Epoch 1/10
750/750 ————— **32s** 41ms/step - accuracy: 0.9347 - loss: 0.2208 - val_accuracy: 0.9731 - val_loss: 0.0976
Epoch 2/10
750/750 ————— **42s** 42ms/step - accuracy: 0.9797 - loss: 0.0680 - val_accuracy: 0.9827 - val_loss: 0.0614
Epoch 3/10
750/750 ————— **30s** 40ms/step - accuracy: 0.9869 - loss: 0.0440 - val_accuracy: 0.9847 - val_loss: 0.0540
Epoch 4/10
750/750 ————— **42s** 42ms/step - accuracy: 0.9904 - loss: 0.0308 - val_accuracy: 0.9812 - val_loss: 0.0649
Epoch 5/10
750/750 ————— **39s** 40ms/step - accuracy: 0.9927 - loss: 0.0232 - val_accuracy: 0.9855 - val_loss: 0.0495
Epoch 6/10
750/750 ————— **30s** 39ms/step - accuracy: 0.9954 - loss: 0.0156 - val_accuracy: 0.9841 - val_loss: 0.0551
Epoch 7/10
750/750 ————— **30s** 40ms/step - accuracy: 0.9965 - loss: 0.0116 - val_accuracy: 0.9827 - val_loss: 0.0619
Epoch 8/10
750/750 ————— **30s** 40ms/step - accuracy: 0.9971 - loss: 0.0092 - val_accuracy: 0.9850 - val_loss: 0.0577
Epoch 9/10
750/750 ————— **43s** 42ms/step - accuracy: 0.9979 - loss: 0.0065 - val_accuracy: 0.9855 - val_loss: 0.0579
Epoch 10/10
750/750 ————— **30s** 40ms/step - accuracy: 0.9980 - loss: 0.0064 - val_accuracy: 0.9854 - val_loss: 0.0634

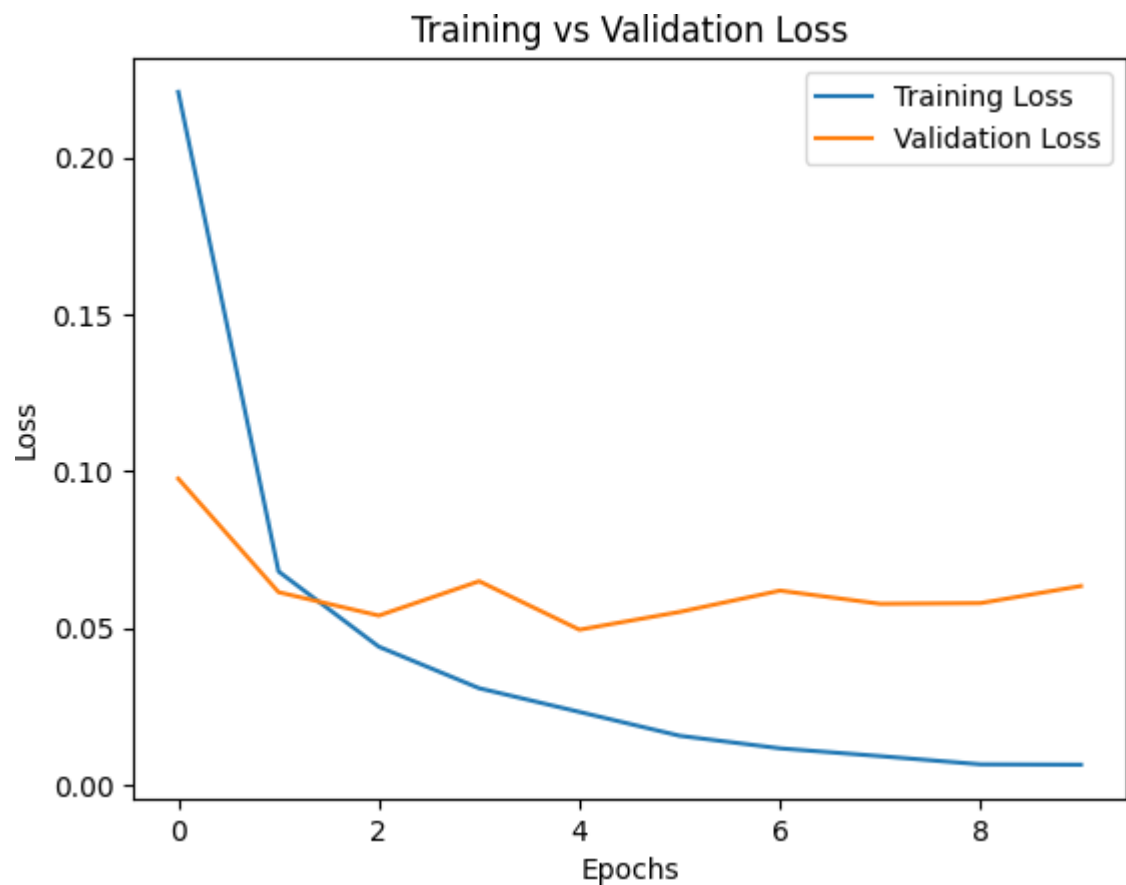
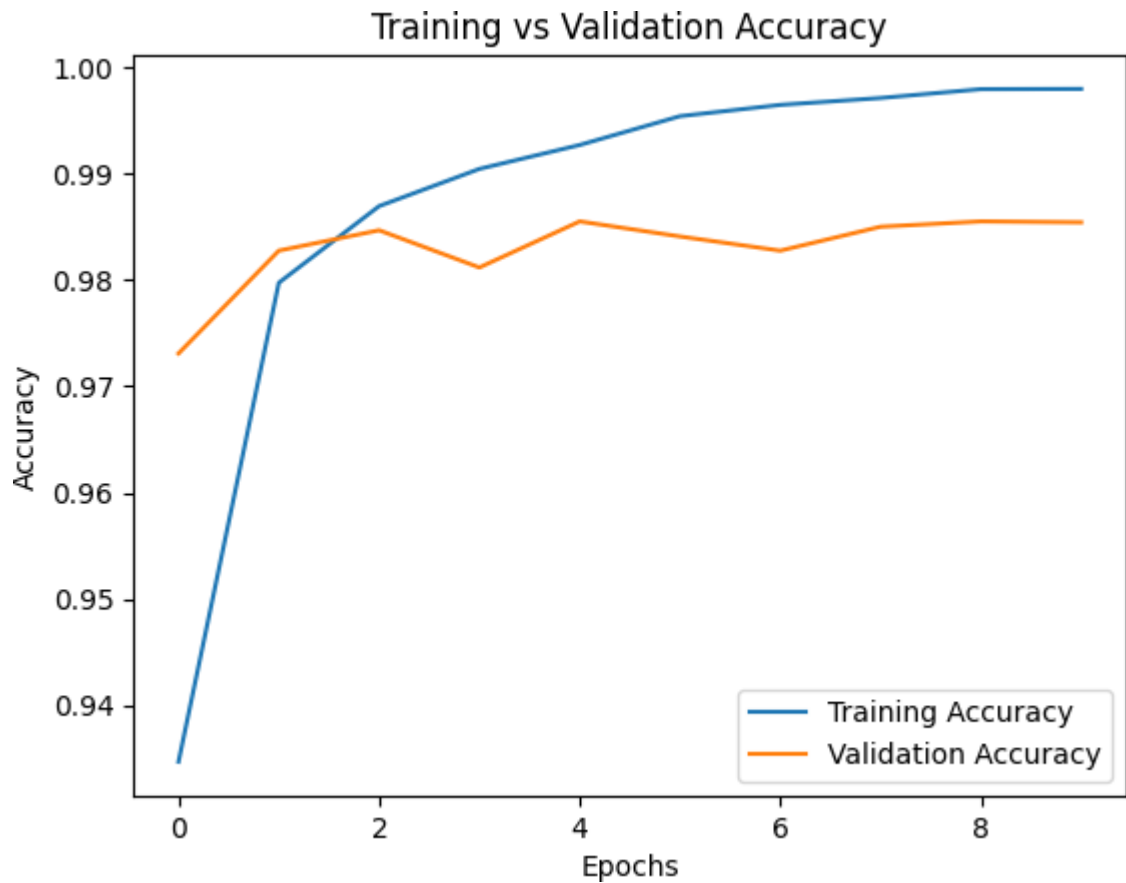
Exercise 7: Model Evaluation

```
In [9]: test_loss, test_accuracy = model.evaluate(x_test, y_test)
        print(f"Test Accuracy: {test_accuracy}")
```

313/313 ————— **2s** 7ms/step - accuracy: 0.9870 - loss: 0.0532
Test Accuracy: 0.9869999885559082

Exercise 8: Visualization of Training Process

```
In [10]: plt.plot(history.history['accuracy'], label='Training Accuracy')
        plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
        plt.title('Training vs Validation Accuracy')
        plt.xlabel('Epochs')
        plt.ylabel('Accuracy')
        plt.legend()
        plt.show()
        plt.plot(history.history['loss'], label='Training Loss')
        plt.plot(history.history['val_loss'], label='Validation Loss')
        plt.title('Training vs Validation Loss')
        plt.xlabel('Epochs')
        plt.ylabel('Loss')
        plt.legend()
        plt.show()
```



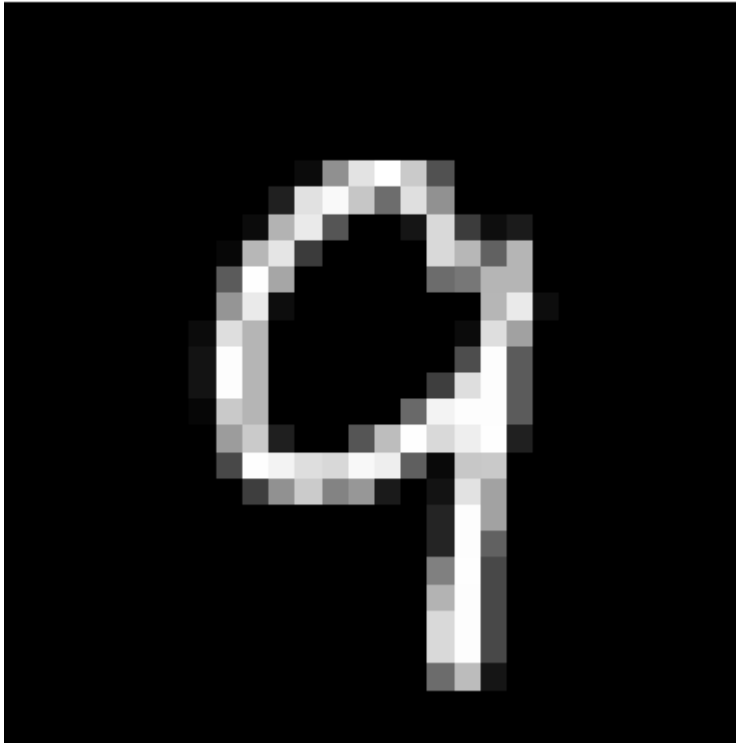
Exercise 9: Model Prediction

```
In [11]: import numpy as np
random_idx = np.random.randint(0, len(x_test))
random_image = x_test[random_idx]
```

```
predicted_class = np.argmax(model.predict(random_image.reshape(1, 28, 28, 1)))
plt.imshow(random_image.reshape(28, 28), cmap='gray')
plt.title(f"Predicted: {predicted_class}")
plt.axis('off')
plt.show()
```

1/1 ————— 0s 133ms/step

Predicted: 9



Exercise 10: Model Modification

```
In [12]: model_modified = Sequential()
model_modified.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1))
model_modified.add(MaxPooling2D((2, 2)))
model_modified.add(Conv2D(64, (3, 3), activation='relu'))
model_modified.add(MaxPooling2D((2, 2)))
model_modified.add(Flatten())
model_modified.add(Dense(128, activation='relu'))
model_modified.add(Dense(10, activation='softmax'))
model_modified.compile(optimizer='adam', loss='categorical_crossentropy', metric
history_modified = model_modified.fit(x_train, y_train, epochs=10, batch_size=64
```

Epoch 1/10
750/750 ————— **50s** 64ms/step - accuracy: 0.9438 - loss: 0.1898 - val_accuracy: 0.9787 - val_loss: 0.0750
Epoch 2/10
750/750 ————— **46s** 62ms/step - accuracy: 0.9842 - loss: 0.0515 - val_accuracy: 0.9861 - val_loss: 0.0468
Epoch 3/10
750/750 ————— **81s** 60ms/step - accuracy: 0.9886 - loss: 0.0356 - val_accuracy: 0.9869 - val_loss: 0.0439
Epoch 4/10
750/750 ————— **46s** 62ms/step - accuracy: 0.9917 - loss: 0.0258 - val_accuracy: 0.9877 - val_loss: 0.0402
Epoch 5/10
750/750 ————— **42s** 56ms/step - accuracy: 0.9936 - loss: 0.0202 - val_accuracy: 0.9887 - val_loss: 0.0391
Epoch 6/10
750/750 ————— **44s** 59ms/step - accuracy: 0.9952 - loss: 0.0145 - val_accuracy: 0.9892 - val_loss: 0.0399
Epoch 7/10
750/750 ————— **45s** 60ms/step - accuracy: 0.9964 - loss: 0.0106 - val_accuracy: 0.9905 - val_loss: 0.0390
Epoch 8/10
750/750 ————— **43s** 58ms/step - accuracy: 0.9967 - loss: 0.0094 - val_accuracy: 0.9898 - val_loss: 0.0410
Epoch 9/10
750/750 ————— **43s** 57ms/step - accuracy: 0.9972 - loss: 0.0079 - val_accuracy: 0.9891 - val_loss: 0.0406
Epoch 10/10
750/750 ————— **43s** 57ms/step - accuracy: 0.9977 - loss: 0.0069 - val_accuracy: 0.9899 - val_loss: 0.0438

Exercise 11: Model Comparison

In [14]: `import pandas as pd`

```
comparison_table = pd.DataFrame({
    "CNN Architecture": ["Basic CNN", "CNN + Extra Conv Layer", "CNN + Dropout",
    "Test Accuracy": [test_accuracy, model_modified.evaluate(x_test, y_test)[1],
    ])

print(comparison_table)
```

313/313 ————— **4s** 13ms/step - accuracy: 0.9901 - loss: 0.0357

	CNN Architecture	Test Accuracy
0	Basic CNN	0.987
1	CNN + Extra Conv Layer	0.9901
2	CNN + Dropout	To be added
3	CNN (More Epochs)	To be added